

CS790 Assignment 3 Detailed Explanation

Title: CS790 Assignment Detailed explanation

Name: Vansil Rakeshkumar Chauhan 24m0847

task_1.py – Detailed Explanation

- This script is a hands-on exercise in manually building and using a **4-hop Tor circuit**, using the Tor ControlPort via **Stem**, and routing HTTP traffic through that circuit.
-

Tor Controller Integration (Stem)

I use the Stem library to directly control the Tor daemon via its ControlPort. The key controller commands used here are:

- `Controller.from_port(port=9051)`
This connects to the running Tor process listening on the default control port.
- `controller.authenticate()`
This authenticates the connection, using either cookie-based or password-based auth (configured in `torrc`).
- `controller.new_circuit(path=[fp1, fp2, fp3, fp4], await_build=True)`
This is the central part: I manually construct a 4-hop circuit by specifying relay fingerprints.

The `await_build=True` flag ensures the function blocks until the circuit is fully built.

- `controller.set_conf('__LeaveStreamsUnattached', '1')`

Normally, Tor automatically attaches new streams to circuits. This config disables that behavior, giving me control over which circuit streams use.

- `controller.add_event_listener(..., EventType.STREAM)`

I register a listener to detect new streams. When one appears, I manually attach it to the custom circuit using:

- `controller.attach_stream(stream.id, circuit_id)`

This is where I force the stream (e.g., a GET request) to use the circuit I built earlier.

- `controller.get_circuits()` and `controller.close_circuit()`

These let me list and clean up circuits, ensuring unused or failed circuits don't linger.

Parsing Relays

This script doesn't parse a separate consensus file. Instead, I use:

```
relays = controller.get_network_statuses().values()
```

This gets all relays Tor currently knows about (based on its own internal consensus). I then filter these relays by their flags (`Guard`, `Exit`, etc.) and apply bandwidth-weighted random selection via a helper:

```
weighted_choice()
```

This approximates Tor's own logic for selecting relays: higher-bandwidth relays are more likely to be picked.

task_2.py – Detailed Explanation

- This script is a more advanced, interactive tool for building and managing **custom Tor circuits** from scratch. It fetches live consensus data, parses it manually, and uses Stem to build n-hop circuits.
-

Consensus Parsing Logic

Unlike `task_1.py`, this script doesn't rely on Tor's internal view of relays. Instead, I:

1. Fetch the latest consensus using:

```
https://collector.torproject.org/recent/relay-descriptors/consensuses/
```

2. Parse the consensus manually using `re` and `base64`:

- Each relay section starts with an `r` line. I extract:
 - Nickname
 - Base64-encoded fingerprint (decoded to hex)
 - IP address
 - ORPort
- I also parse the `s` line for relay flags (`Running`, `Guard`, `Exit`, etc.)
- The `w` line gives the advertised bandwidth

3. Each relay becomes a `Relay` object, which I store in a list for selection.

This lets me work entirely from the live, publicly available consensus data — giving me independence from Tor's internal state.

Tor Controller Integration (Stem)

Once I have the relays, I use the same Stem commands as before to control Tor:

- `Controller.from_port()` / `authenticate()`
Standard setup to connect and authenticate with the Tor daemon.
 - `controller.new_circuit(path=[...])`
Builds a custom circuit with any number of hops (user-defined).
 - `controller.set_conf('__LeaveStreamsUnattached', '1')`
Turns off Tor's automatic stream handling.
 - `controller.add_event_listener(..., EventType.STREAM)`
Used to detect new streams and attach them to the selected circuit.
 - `controller.attach_stream()`
Forces the stream to use the circuit I built.
 - `controller.get_circuits()`
Used to list all available circuits for interaction.
 - `controller.close_circuit()`
Allows the user to manually close circuits.
 - `controller.reset_conf()` and `remove_event_listener()`
Resets Tor's configuration to its default behavior after stream control is no longer needed.
-

Interactive Features

The script has a menu that supports:

- Creating circuits of user-defined length
 - Listing and managing circuits
 - Attaching HTTP requests to circuits
 - Resetting Tor's stream handling
 - Sending test HTTP requests through Tor
-